UNIVERSITÀ DEGLI STUDI DI BARI ALDO MORO | DIPARTIMENTO DI INFORMATICA

Documentazione Caso di Studio
Ingegneria della Conoscenza
A.A. 2025/26
Logistica Flotta Furgoni Bari

Gruppo di lavoro

- Michele Barone, 760395, m.barone23@studenti.uniba.it

Repository GitHub

- https://github.com/Michele5576/ICON_LOGISTICA

Sommario

Introduzione 3
Dati di partenza 3
Sommario 3
Elenco argomenti di interesse 4
Rappresentazione e ragionamento: la Knowledge Base 5
Sommario 5
Strumenti utilizzati 6
Decisioni di Progetto 6
Valutazione 7
Apprendimento supervisionato 8
Sommario 8
Strumenti utilizzati 8
Decisioni di Progetto 8
Valutazione 9
Ragionamento con vincoli: assegnamento clienti-veicoli 11
Sommario 11
Strumenti utilizzati 11
Decisioni di Progetto 11
Valutazione 12
Ricerca: ordine di visita e percorsi stradali 15
Sommario 15
Strumenti utilizzati 15
Decisioni di Progetto 15
Valutazione 15
Valutazione del sistema integrato 17
Conclusione 18
Riferimenti Bibliografici 19

Introduzione

Si considera un'azienda di distribuzione alimentare B2B che rifornisce ristoranti, bar e botteghe (canale HORECA) con una flotta interna di furgoni, in parte refrigerati. Con il rincaro del carburante degli ultimi tempi il costo di ogni chilometro percorso pesa in modo crescente sul margine: diventa quindi importante pianificare i giri giornalieri in modo da usare meno mezzi e meno strada, senza mancare le finestre orarie richieste dai clienti.

Il caso di studio affronta questo problema per la città di Bari, dove la pianificazione è resa più difficile da vincoli che un semplice calcolo delle distanze non vede: la zona a traffico limitato di Murat e Bari Vecchia, i permessi riservati ai mezzi più piccoli, i giorni festivi e gli eventi cittadini come il corteo storico di San Nicola, durante i quali il centro è chiuso quasi del tutto. A questi si aggiunge il comportamento dei clienti osservato nello storico: alcuni richiedono sistematicamente tempi di scarico più lunghi, altri risultano spesso assenti al passaggio del furgone.

Il sistema realizzato è un KBS che, data una data di pianificazione e l'elenco dei clienti da servire, decide quale furgone serve ciascun cliente, in che ordine e lungo quale percorso, e spiega perché un cliente non può essere servito. Integra quattro componenti che si scambiano conoscenza (Figura 1): una base di conoscenza logica, modelli appresi dallo storico, un problema di soddisfacimento di vincoli e algoritmi di ricerca su grafo.

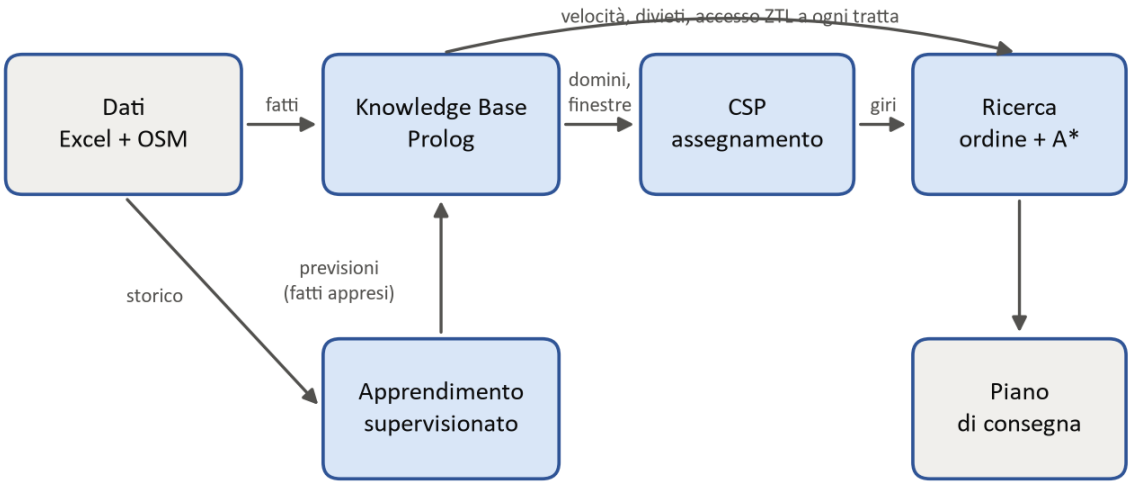


Figura 1: architettura del sistema e flussi di conoscenza tra i moduli.

Dati di partenza

I dati sono un file Excel con tre fogli (flotta di 5 furgoni, anagrafica di 100 clienti con finestra oraria richiesta, storico di 1393 consegne in 26 giornate lavorative di giugno 2026) e l'estratto OpenStreetMap[1] della città di Bari (57.242 nodi e 13.054 strade). Le anagrafiche del dataset sono fittizie: ragioni sociali generiche e indirizzi che ricorrono identici in città diverse. I clienti sono distribuiti su sette città; il sistema pianifica i 33 clienti di Bari, l'area coperta dalla mappa, mentre lo storico di tutti i 100 clienti è usato per l'apprendimento.

Sommario

La pipeline (`src/main.py`) esegue, per ogni data di pianificazione, quattro passi:

- **Knowledge Base Prolog**: deduce la classe di ogni veicolo, il tipo di giorno, gli intervalli in cui ciascun veicolo può raggiungere ciascun cliente e i vincoli di transito sulle strade; produce l'albero di prova delle proprie conclusioni e il motivo di ogni esclusione.
- **Apprendimento supervisionato**: due modelli stimano, per ogni cliente, i minuti di scarico oltre lo standard e la probabilità di mancata consegna; le stime entrano nella KB come fatti, da cui la KB deriva la durata del servizio e i clienti da preavvisare.
- **CSP con vincoli flessibili**: assegna i clienti ai veicoli usando i domini, le finestre e le durate fornite dalla KB; è risolto in modo esatto con Branch and Bound e consistenza d'arco, e in modo approssimato con Simulated Annealing.
- **Ricerca**: per ogni veicolo cerca l'ordine di visita migliore e calcola ogni tratta con A* sul grafo stradale, interrogando la KB sull'accesso alla ZTL all'orario di partenza.

Elenco argomenti di interesse

Gli argomenti sono tratti da capitoli diversi del programma; si indicano i capitoli delle dispense del corso[2] e le sezioni corrispondenti del testo di riferimento[3].

- **Rappresentazione e ragionamento** (capp. 4 e 5: ragionamento proposizionale e relazionale; cap. 6: knowledge graph e ontologie): clausole definite Datalog/Prolog, gerarchie di classi con chiusura transitiva ed ereditarietà, default ed eccezioni con negazione come fallimento, vincoli d'integrità, debugging a livello di conoscenza tramite alberi di prova.
- **Apprendimento supervisionato** (cap. 7): regressione e classificazione con modelli lineari regolarizzati, alberi di decisione, bagging (Random Forest) e boosting; pseudoconteggi; valutazione con cross-validation annidata ripetuta.
- **Ragionamento con vincoli** (cap. 3): CSP con vincoli rigidi binari e n-ari e vincoli flessibili, consistenza d'arco generalizzata, separazione dei domini, Branch and Bound, ricerca locale stocastica.
- **Ricerca nello spazio degli stati** (cap. 2): A* con euristica ammissibile e confronto con la ricerca a costo minimo, potatura dei cammini multipli, Branch and Bound in profondità.

Rappresentazione e ragionamento: la Knowledge Base

Sommario

La KB (`kb/knowledge_base.pl`) contiene la conoscenza di dominio necessaria a decidere chi può servire chi e quando. Separa i fatti, generati automaticamente dai dati, dalle regole, scritte a mano: 39 predicati, 58 regole e 118 fatti di dominio (tassonomie, velocità, calendario), più 2.456 fatti generati. I fatti generati descrivono i veicoli (`veicolo/7`), i clienti (`cliente/3`), le finestre richieste in minuti dalla mezzanotte (`finestra/3`), il nodo stradale di ogni cliente (`ubicazione/2`), i 2.152 nodi che cadono nella ZTL (`in_ztl/1`) e le previsioni del modulo di apprendimento (`scarico_previsto/2`, `prob_mancata/2`).

Le regole su classi, permessi, orari e idoneità stanno solo nella KB. Nella prima versione del progetto il permesso ZTL veniva scritto dal parser Python (portata ≤ 800 kg) e la KB si limitava a confrontare orari: era di fatto un database. Ora il permesso è dedotto dalla classe del veicolo, e Python calcola soltanto dati geometrici (quali nodi cadono nel rettangolo della ZTL, posizione dei clienti).

Classi, proprietà ed ereditarietà

I veicoli sono organizzati in una gerarchia di classi[4]: furgone compatto e furgone grande sono sottoclassi di veicolo commerciale leggero, a sua volta veicolo commerciale e veicolo. La relazione `is_a/2` ne è la chiusura transitiva, definita ricorsivamente in stile Datalog[5], e `istanza/2` propaga l'appartenenza alle superclassi. La classe di un veicolo non è scritta nei dati ma dedotta dai suoi attributi (portata e volume), e da essa discende il permesso ZTL (Tabella 1).

Tabella 1: veicoli della flotta, classe dedotta dalla KB e permesso derivato.

Veicolo	Modello	Portata (kg)	Volume (m³)	Cella frigo	Classe (dedotta)	Permesso ZTL
v_spr_01 … v_spr_04	Mercedes Sprinter	1500	14	sì	furgone_grande	no
v_cit_05	Mercedes Citan	700	3,8	no	furgone_compatto	sì

Anche le strade hanno una tassonomia: i 14 tipi OSM percorribili sono raggruppati in quattro categorie (scorrimento, quartiere, locale, ristretta). Velocità e divieti seguono lo schema dei default con eccezioni[6]: la velocità base vale per ogni veicolo, salvo che non sia derivabile una limitazione per la sua classe; i furgoni grandi non possono attraversare le living_street, salvo per raggiungere la destinazione.

```
velocita(V, Tipo, Kmh) :-
    veicolo(V, _, _, _, _, _, _), velocita_base(Tipo, Kmh), \+ limitazione(V, Tipo, _).
velocita(V, Tipo, Kmh) :-
    velocita_base(Tipo, K0), limitazione(V, Tipo, K1), Kmh is min(K0, K1).
limitazione(V, Tipo, 25) :- istanza(V, furgone_grande), categoria(Tipo, locale).
limitazione(V, Tipo, 15) :- istanza(V, furgone_grande), categoria(Tipo, ristretta).
```

Calendario e ZTL

Il tipo di giorno è dedotto dalla data: festività nazionali e patronale (6 dicembre), domeniche (calcolate con `day_of_the_week/2`) ed eventi cittadini. Una data può essere sia festiva sia di evento: la priorità evento > festivo > feriale è espressa con la negazione come fallimento, in modo che ogni data abbia un solo tipo.

```
tipo_giorno(data(G, M, _), evento) :- evento_cittadino(_, G, M).
tipo_giorno(D, festivo) :- \+ tipo_giorno(D, evento), festivo(D).
tipo_giorno(D, feriale) :- \+ tipo_giorno(D, evento), \+ festivo(D).
```

Dal tipo di giorno dipendono gli orari della ZTL e la validità del permesso (Tabella 2): durante gli eventi la ZTL è attiva tutto il giorno e il permesso vale solo nella fascia di carico e scarico. Le fasce sono un'approssimazione dichiarata delle regole reali, e la ZTL è rappresentata come un rettangolo che copre Murat e Bari Vecchia.

Tabella 2: orari della ZTL e validità del permesso per tipo di giorno.

Tipo di giorno	Esempio	ZTL attiva	Accesso con permesso
feriale	mercoledì 1/7/2026	07:30-20:00	sempre
festivo	domenica, 2 giugno	10:00-23:00	sempre
evento	7-8 maggio (San Nicola)	tutto il giorno	solo 07:00-10:00 (carico/scarico)

Finestre di consegna effettive e conoscenza appresa

La regola centrale della KB interseca tre intervalli: la finestra richiesta dal cliente, gli intervalli in cui il veicolo può trovarsi nel punto di consegna (che dipendono da ZTL, permesso e tipo di giorno) e il turno di lavoro 08:00-17:00. Un veicolo è idoneo per un cliente in una data se l'intersezione non è vuota; le finestre risultanti diventano i dati del CSP.

```
finestra_effettiva(V, C, D, I, F) :-
    veicolo(V, _, _, _, _, _, _), finestra(C, CI, CF), turno_lavoro(TI, TF),
    intervallo_accesso(V, C, D, AI, AF),
    I is max(CI, max(AI, TI)), F is min(CF, min(AF, TF)), I < F.

intervallo_accesso(V, C, D, I, F) :-          % giorno di evento, veicolo con permesso
    cliente_in_ztl(C), permesso_ztl(V),
    tipo_giorno(D, evento), fascia_carico_scarico(I, F).
```

Le previsioni dell'apprendimento entrano come fatti: `durata_servizio/2` usa la previsione se esiste e altrimenti il default di 10 minuti (ancora negazione come fallimento), mentre `richiede_preavviso/1` segnala i clienti con probabilità di mancata consegna almeno 0,5.

Alberi di prova, motivi di fallimento e vincoli d'integrità

Il predicato `dimostra/2` percorre le clausole usate per dimostrare un goal (tramite `clause/2`) e restituisce l'albero di prova: quali fatti e quali regole hanno portato alla conclusione. È lo strumento del debugging a livello di conoscenza[7]: davanti a una risposta errata si risale alla clausola responsabile. Per i fallimenti, `perche_non_idoneo/4` indica la causa: cliente fuori area, finestra fuori dal turno, ZTL senza permesso, permesso sospeso per evento.

I vincoli d'integrità[8] sono regole `violazione/1` derivabili solo se la KB è incoerente: finestre vuote, clienti senza finestra, veicoli non classificabili o con due classi, ubicazioni fuori area o multiple, tipi di strada senza categoria, probabilità fuori da [0, 1].

Strumenti utilizzati

SWI-Prolog 10[9] come motore di inferenza e PySwip[10] come interfaccia da Python. Tutte le interrogazioni passano dalla classe `KnowledgeBase` (`src/kb_interface/prolog_interface.py`): gli altri moduli non scrivono codice Prolog. I fatti sono generati dagli script in `src/parsing/` a partire dall'Excel e dall'XML OSM.

Decisioni di Progetto

- **Grafo stradale fuori dalla KB.** Il grafo è esportato anche in Prolog (`kb/grafo.pl`, circa 92.000 fatti `nodo/3` e `arco/3`), ma non viene caricato: la ricerca dei percorsi su un grafo di queste dimensioni è affidata ad A* in Python, a cui la KB fornisce velocità, divieti e accesso alla ZTL di ogni veicolo.

- **Posizione dei clienti.** La geocodifica degli indirizzi fittizi portava i 33 clienti di Bari su soli 11 nodi, spesso ai bordi della mappa. Ogni cliente di Bari è quindi collocato su un nodo stradale reale della propria macrozona (Bari Città: Murat, Madonnella, Bari Vecchia; Bari Nord: Libertà, Marconi, San Girolamo; Bari Sud: Carrassi, Poggiofranco, Japigia), su una strada adatta alla consegna, con una scelta pseudo-casuale riproducibile (seed = codice cliente). Ne risultano 4 clienti in ZTL.
- **Tempo in minuti e intervalli semiaperti.** Tutti gli orari sono interi (minuti dalla mezzanotte) e gli intervalli sono [inizio, fine): l'intersezione si calcola con max e min, senza enumerare gli istanti.
- **Clienti come fatti dinamici.** `cliente/3`, `finestra/3` e `ubicazione/2` sono dichiarati dinamici: nuovi clienti o ordini possono essere aggiunti a runtime senza rigenerare i file.

Valutazione

La correttezza della KB è verificata da 12 test automatici (`tests/test_kb.py`, parte dei 24 test del progetto): classificazione ed ereditarietà, eccezioni sulle velocità, priorità del tipo di giorno (il 7 maggio 2027 è un evento anche se cade di venerdì; il 2 giugno e le domeniche sono festivi), accesso alla ZTL in cinque situazioni, finestre effettive contenute nel turno e nella finestra richiesta, motivi di fallimento (compreso un cliente fittizio con finestra serale), integrazione delle previsioni e coerenza del turno con la configurazione Python. Un test inserisce di proposito un fatto errato (probabilità 1,5) e verifica che il vincolo d'integrità lo rilevi; sulla KB reale nessun vincolo è violato.

Per stimare il costo del ragionamento si è misurato il numero di inferenze logiche delle interrogazioni principali (Tabella 3).

Tabella 3: inferenze logiche per le interrogazioni principali (statistics/2).

Query	Inferenze
tipo_giorno(data(7,5,2027), T)	11
veicoli_idonei(cliente ZTL, feriale)	1628
idoneo(V, C, feriale) per tutti V, C	4348
finestra_effettiva(V, C, evento, I, F)	10499
dimostra(idoneo(...)) (albero di prova)	1645
violazione(X) (tutti i vincoli)	492

Il costo è dominato da `finestra_effettiva/5`, che cresce con il numero di veicoli, clienti e intervalli di accesso: calcolare tutte le finestre di una giornata di evento per 5 veicoli e 33 clienti richiede circa 10.500 inferenze, un tempo trascurabile rispetto alla ricerca. Un esempio di albero di prova, abbreviato:

```
?- dimostra(idoneo(v_cit_05, c_970855, data(1,7,2026)), P).
regola(idoneo(...),
  regola(servibile(c_970855), fatto(ubicazione(c_970855, n1437608842)))
  regola(finestra_effettiva(v_cit_05, c_970855, data(1,7,2026), 660, 780),
    fatto(finestra(c_970855, 660, 780))   fatto(turno_lavoro(480, 1020))
    regola(intervallo_accesso(v_cit_05, c_970855, data(1,7,2026), 450, 1200),
      regola(cliente_in_ztl(c_970855), ... fatto(in_ztl(n1437608842)))
      regola(permesso_ztl(v_cit_05), regola(istanza(v_cit_05, furgone_compatto), ...))
      regola(tipo_giorno(data(1,7,2026), feriale),
        non(tipo_giorno(data(1,7,2026), evento)), non(festivo(data(1,7,2026))))
      fatto(fascia_ztl(feriale, 450, 1200)))
    sistema(660 is max(660, max(450, 480)))  sistema(780 is min(780, min(1200, 1020))) ...))
```

Nello scenario di evento la KB esclude i 4 clienti in ZTL e motiva l'esclusione veicolo per veicolo: `ztl_senza_permesso` per gli Sprinter e `permesso_sospeso_per_evento` per il Citan, perché le finestre richieste da quei clienti non cadono nella fascia 08:00-10:00 (carico e scarico intersecato con il turno).

Apprendimento supervisionato

Sommario

Dallo storico delle consegne si apprendono due funzioni, usate come conoscenza nella pianificazione: (R) i minuti di scarico oltre i 10 previsti, una regressione sulle 1309 consegne completate; (C) la mancata consegna per cliente assente, una classificazione binaria sulle 1393 consegne, con il 6% di casi positivi. L'unità di analisi è la singola consegna.

Nella prima versione del progetto si classificava un "profilo" per cliente (100 esempi) il cui target era calcolato con una regola fissa a partire dalle stesse feature date al modello: l'accuratezza di 0,99 misurava soltanto la capacità di reimparare quella regola. Il dataset è stato quindi ricostruito da capo, con particolare attenzione a usare solo informazione disponibile al momento della decisione.

Strumenti utilizzati

scikit-learn[11] per modelli, ricerca degli iperparametri e metriche, pandas per la preparazione dei dati. Sono confrontati una baseline costante (media, oppure classe estratta con le frequenze osservate), un modello lineare regolarizzato (Ridge per R, regressione logistica L2 per C), un albero di decisione, Random Forest[12] e Gradient Boosting[13], i modelli composti trattati nel programma[14].

Decisioni di Progetto

Le feature sono divise in tre gruppi a seconda di quando sono note (Tabella 4).

Tabella 4: feature del dataset di apprendimento.

Gruppo	Feature	Quando è nota	Uso
Storico del cliente	n. consegne precedenti, extra medio e massimo, frequenza di scarichi lunghi, frequenza di mancate consegne	prima della pianificazione	modello finale
Contesto	giorno della settimana, inizio e ampiezza della finestra, area (4 one-hot)	prima della pianificazione	modello finale
Operative	ora di arrivo, posizione nel giro, km stimati, tipo di veicolo	solo a giro effettuato	solo ablazione

- **Nessun leakage.** Per la consegna del giorno d, le feature storiche del cliente sono calcolate solo sulle consegne dei giorni precedenti (finestra espandente): è esattamente l'informazione che si avrebbe pianificando quel giorno. Le feature operative, note solo a giro effettuato, sono escluse dal modello finale e usate solo per un'ablazione.
- **Pseudoconteggi**[15]. Le medie storiche sono smussate verso la media globale (anch'essa calcolata sul passato) con 2 consegne virtuali: un cliente senza storico riceve il valore globale invece di 0, e una singola osservazione non porta la stima agli estremi.
- **Protocollo di valutazione.** Cross-validation annidata: 5 fold esterni raggruppati per data (tutte le consegne di uno stesso giorno stanno nello stesso fold, così si valuta su giornate mai viste), ripetuti 3 volte rimescolando le giornate, per un totale di 15 test set; nei fold interni (3, sempre raggruppati per data) GridSearchCV sceglie gli iperparametri, ottimizzando il MAE per R e l'F1 della classe positiva per C. Si riportano media e deviazione standard sui 15 test set.
- **Costo degli errori in C.** Un cliente assente non previsto (falso negativo) costa più di una telefonata di preavviso inutile (falso positivo): dove il modello lo consente, gli errori sulla classe rara sono pesati

in proporzione inversa alla sua frequenza. Si riportano precision, recall, F1 e area sotto la curva ROC[16] invece dell'accuratezza, che premierebbe il classificatore che risponde sempre "consegnata" (94%).

- **Iperparametri esplorati.** Ridge $\alpha \in \{0{,}1; 1; 10; 100\}$; logistica $C \in \{0{,}01; 0{,}1; 1; 10\}$; albero profondità $\in \{3, 5, 8\}$ e foglie minime $\in \{5, 20\}$; Random Forest (200 alberi) profondità $\in \{5, 10, \text{illimitata}\}$ e foglie minime $\in \{1, 5\}$; Gradient Boosting stimatori $\in \{100, 200\}$, learning rate $\in \{0{,}05; 0{,}1\}$, profondità $\in \{2, 3\}$. Il modello scelto per ciascun task è riaddestrato su tutti i dati con la stessa ricerca.

Valutazione

Tabella 5: regressione dei minuti di scarico extra (media ± dev. std. su 15 test set).

Modello	MAE	RMSE
Baseline (media)	6.910 ± 0.337	9.141 ± 0.710
Ridge	2.709 ± 0.439	4.646 ± 1.154
Albero di decisione	2.070 ± 0.466	4.464 ± 1.101
Random Forest	2.009 ± 0.473	4.338 ± 1.151
Gradient Boosting	2.086 ± 0.489	4.285 ± 1.185

Tabella 6: classificazione della mancata consegna (media ± dev. std. su 15 test set).

Modello	Precision	Recall	F1	AUC
Baseline (stratificata)	0.052 ± 0.058	0.058 ± 0.077	0.053 ± 0.063	0.499 ± 0.039
Regressione logistica	0.421 ± 0.082	0.870 ± 0.108	0.561 ± 0.074	0.937 ± 0.019
Albero di decisione	0.341 ± 0.112	0.900 ± 0.114	0.478 ± 0.114	0.901 ± 0.049
Random Forest	0.386 ± 0.072	0.908 ± 0.103	0.536 ± 0.071	0.949 ± 0.013
Gradient Boosting	0.424 ± 0.163	0.313 ± 0.129	0.338 ± 0.094	0.934 ± 0.030

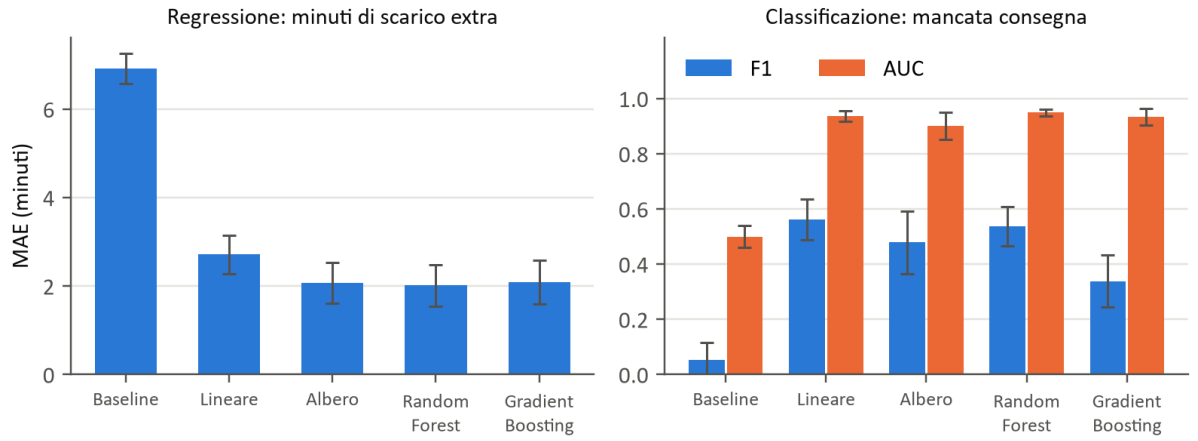


Figura 2: confronto dei modelli; le barre d'errore sono la deviazione standard sui 15 test set.

Tutti i modelli superano nettamente la baseline (Tabelle 5 e 6, Figura 2). Per la regressione gli ensemble sono equivalenti entro la deviazione standard; è stato scelto Random Forest, con l'errore medio più basso (2,0 minuti contro i 6,9 della baseline) e profondità massima 5, il valore scelto in 9 fold su 15. Il modello lineare resta indietro perché la relazione tra storico e scarico non è lineare: molti clienti non sforano mai, pochi sforano di molto.

Per la classificazione la regressione logistica fortemente regolarizzata ($C = 0{,}01$, scelto in 14 fold su 15) ha l'F1 migliore (0,56) e un'AUC di 0,94. Random Forest ordina i casi in modo appena migliore (AUC 0,95) ma alla soglia 0,5 ha una precision più bassa; Gradient Boosting, che non pesa gli errori sulla classe rara, ordina bene (AUC 0,93) ma alla stessa soglia riconosce meno di un assente su tre. La deviazione standard tra i fold è legata al numero ridotto di positivi per fold (circa 17).

Per capire da dove viene il segnale, il modello migliore di ciascun task è stato rivalutato con tre insiemi di feature (Tabelle 7 e 8).

Tabella 7: ablazione sui gruppi di feature, regressione (Random Forest).

Feature usate	MAE	RMSE
solo contesto	4.165 ± 0.383	7.191 ± 0.603
storico + contesto (pianificazione)	2.009 ± 0.473	4.338 ± 1.151
pianificazione + operative	2.016 ± 0.461	4.347 ± 1.107

Tabella 8: ablazione sui gruppi di feature, classificazione (regressione logistica).

Feature usate	Precision	Recall	F1	AUC
solo contesto	0.070 ± 0.015	0.574 ± 0.125	0.124 ± 0.024	0.622 ± 0.062
storico + contesto (pianificazione)	0.421 ± 0.082	0.870 ± 0.108	0.561 ± 0.074	0.937 ± 0.019
pianificazione + operative	0.415 ± 0.074	0.866 ± 0.120	0.555 ± 0.072	0.932 ± 0.024

Il segnale sta nello storico del cliente: senza le feature storiche il MAE raddoppia, l'F1 scende a 0,12 e l'AUC a 0,62. Le feature operative non aggiungono nulla, il che conferma che escluderle non costa accuratezza. Il contesto da solo conserva un po' di informazione perché finestra richiesta e area identificano in parte il cliente.

Ragionamento con vincoli: assegnamento clienti-veicoli

Sommario

Decidere quale veicolo serve ciascun cliente è un problema combinatorio: con 18 clienti e 5 veicoli gli assegnamenti possibili sono oltre 10^12. È stato modellato come CSP con vincoli flessibili, ovvero come problema di ottimizzazione vincolata[17] (Tabella 9).

Tabella 9: il CSP di assegnamento.

Elemento	Definizione
Variabili	un cliente da servire nella giornata
Domini	veicoli idonei derivati dalla KB (permessi, ZTL, eventi, finestre, turno)
H1 (rigido, binario)	due clienti sullo stesso veicolo devono ammettere un ordine di visita che rispetti entrambe le finestre effettive
H2 (rigido, n-ario)	per ogni veicolo, somma di servizio previsto e tempo minimo di arrivo ≤ 540 minuti
S1 (flessibile)	distanza stimata tra ogni coppia di clienti dello stesso veicolo (km)
S2 (flessibile)	15 km equivalenti per ogni veicolo utilizzato

I domini arrivano già ridotti dalla KB: un cliente in ZTL ha nel dominio solo i veicoli che possono raggiungerlo in un orario compatibile con la sua finestra. È una consistenza di nodo ottenuta per ragionamento invece che per filtraggio. I vincoli flessibili favoriscono giri geograficamente compatti con pochi mezzi, cioè meno chilometri e meno carburante; il costo di attivazione di un veicolo bilancia la tendenza a dividere i clienti su tutti i furgoni.

Strumenti utilizzati

Gli algoritmi sono implementati in Python (`src/csp/assignment.py`) seguendo il cap. 3 del programma: consistenza d'arco generalizzata[18][19], separazione dei domini[20], Branch and Bound in profondità e Simulated Annealing[21][22]. La prima versione usava il solver CP-SAT di OR-Tools, sostituito per poter controllare, misurare e discutere gli algoritmi del programma.

Decisioni di Progetto

- **H2 come rilassamento.** La durata esatta di un giro dipende dall'ordine di visita, cioè da un problema del commesso viaggiatore con finestre temporali. Per raggiungere un cliente serve almeno il tempo dal punto più vicino (altro cliente o deposito): H2 è quindi un limite inferiore della durata reale e non esclude mai soluzioni ammissibili. L'ammissibilità effettiva è misurata a valle, simulando i giri con A*.
- **Branch and Bound con consistenza.** I nodi dello spazio di ricerca sono assegnazioni parziali. A ogni passo il dominio di una variabile è separato in un caso per valore e GAC propaga la scelta (schema Con_Solve), togliendo il veicolo dai domini dei clienti incompatibili; H2 è verificato sul veicolo scelto. Le variabili sono considerate in ordine decrescente di carico. Il costo di una coppia si somma quando entrambi i clienti sono assegnati, e il bound aggiunge il minimo costo che ogni cliente ancora libero dovrà pagare rispetto ai già assegnati: poiché coppie tra clienti liberi e attivazioni hanno costo non negativo, il bound è ammissibile e la soluzione restituita è ottima se la ricerca termina.
- **Simulated Annealing.** Stato: un assegnamento completo nei domini. Mossa: un cliente scelto a caso passa a un altro veicolo del suo dominio. Funzione da minimizzare: costo dei vincoli flessibili + 1000 per ogni vincolo rigido violato, un costo elevato rispetto ai costi flessibili delle istanze (sempre sotto 250). Una mossa peggiorativa Δ è accettata con probabilità e^(−Δ/T), con raffreddamento

geometrico; il costo è aggiornato in modo incrementale (O(n) per mossa) per poter eseguire 20.000 mosse in pochi decimi di secondo.

- **Strategia di pianificazione.** Prima si esegue il Simulated Annealing, il cui costo diventa il limite iniziale del Branch and Bound (che altrimenti partirebbe da infinito): la ricerca resta esatta ma pota prima. Il tempo massimo è 20 secondi, dopo i quali si usa la migliore soluzione trovata.
- **Distanze stimate.** Distanza in linea d'aria × 1,4, la mediana del rapporto tra distanza stradale e linea d'aria misurato con A* (capitolo successivo), e velocità media urbana di 25 km/h.

Valutazione

Le istanze sono le 26 giornate dello storico, ciascuna con i clienti di Bari serviti quel giorno (da 2 a 18, in media 12,5) e con le previsioni calcolate dallo storico precedente. Il Branch and Bound ha un tempo massimo di 10 secondi; il Simulated Annealing è eseguito con 5 seed per giornata (Tabella 10).

Tabella 10: assegnamento sulle 26 giornate reali ('chiuso' = ricerca completata, ottimo dimostrato).

Algoritmo	Costo	Nodi	Tempo (s)	Esito
B&B (GAC)	78.06 ± 22.78	158643.58 ± 106019.41	6.78 ± 4.42	10/26 chiusi
B&B (GAC) + limite iniziale da SA	75.85 ± 20.49	145482.62 ± 102671.94	6.89 ± 4.53	10/26 chiusi
Simulated Annealing (5 seed)	75.92 ± 20.54	20000 mosse	0.20 ± 0.04	130/130 ammissibili; gap 0.08 ± 0.47 %

Il Branch and Bound completa la ricerca, dimostrando l'ottimo, in 10 giornate su 26; nelle altre si ferma al tempo massimo. Con il limite iniziale fornito dal Simulated Annealing espande meno nodi e restituisce il costo medio più basso, mentre senza resta su soluzioni peggiori nelle giornate interrotte (costo medio 78,1 contro 75,9). Il Simulated Annealing trova sempre assegnamenti ammissibili, in media a 0,08% dalla migliore soluzione nota, in 0,2 secondi contro i 6,9 del Branch and Bound. La lentezza del metodo esatto è dovuta anche agli assegnamenti equivalenti: i quattro Sprinter sono interscambiabili, quindi ogni soluzione compare fino a 24 volte nello spazio di ricerca. GAC non rimuove valori prima della ricerca, perché con un turno di 9 ore e finestre di 2-4 ore due clienti sono quasi sempre compatibili sullo stesso veicolo; durante la ricerca propaga ogni scelta e scarta subito i rami con un dominio vuoto.

La scalabilità è stata misurata su sottoinsiemi casuali dei 33 clienti di Bari (Tabella 11, Figura 3).

Tabella 11: scalabilità al crescere dei clienti (Branch and Bound con limite iniziale dal Simulated Annealing, tempo massimo 10 s).

Clienti	Istanze	B&B tempo (s)	B&B nodi	B&B chiusi	SA tempo (s)	Δ costo SA vs B&B (%)
8	5	0.12 ± 0.15	5044.40 ± 6187.97	5/5	0.17 ± 0.01	0.00 ± 0.00
12	5	8.15 ± 2.64	214417.60 ± 68550.94	2/5	0.20 ± 0.01	0.00 ± 0.00
16	5	10.00 ± 0.00	202335.20 ± 13650.91	0/5	0.23 ± 0.00	0.00 ± 0.00
20	5	10.00 ± 0.00	140707.00 ± 9992.60	0/5	0.27 ± 0.00	0.00 ± 0.00
25	5	10.00 ± 0.00	98178.40 ± 2202.63	0/5	0.32 ± 0.00	0.00 ± 0.00
33	1	10.00 ± 0.00	67079.00 ± 0.00	0/1	0.41 ± 0.00	0.00 ± 0.00

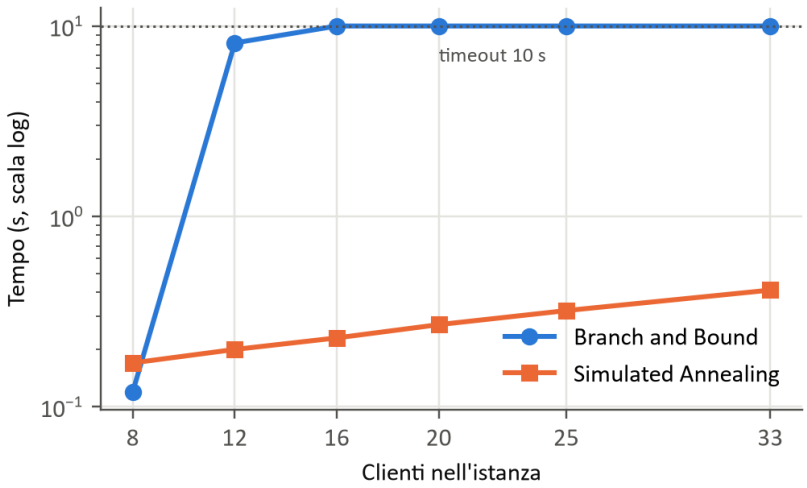


Figura 3: tempo di risoluzione al crescere dei clienti (scala logaritmica).

Il tempo del Branch and Bound cresce in modo esponenziale: chiude tutte le istanze da 8 clienti e 2 su 5 da 12; da 16 clienti in poi raggiunge sempre il tempo massimo senza migliorare la soluzione del Simulated Annealing. Il tempo del Simulated Annealing cresce invece linearmente, da 0,17 a 0,41 secondi. Da qui la strategia adottata: il Simulated Annealing produce la soluzione, il Branch and Bound la migliora o ne dimostra l'ottimalità quando l'istanza è abbastanza piccola.

I parametri del Simulated Annealing sono stati scelti con una taratura su 6 giornate, 3 seed ciascuna (Tabella 12, Figura 4).

Tabella 12: taratura del Simulated Annealing: gap dal costo di riferimento del Branch and Bound.

T iniziale	alfa	Gap (%)	Volte al riferimento	Prove
5.0	0.999	0.10 ± 0.28	15	18
5.0	0.9995	0.10 ± 0.28	15	18
5.0	0.9999	0.00 ± 0.00	18	18
50.0	0.999	0.36 ± 0.74	13	18
50.0	0.9995	0.09 ± 0.26	16	18
50.0	0.9999	2.23 ± 2.92	9	18
500.0	0.999	0.34 ± 0.75	13	18
500.0	0.9995	0.19 ± 0.32	11	18
500.0	0.9999	4.44 ± 5.62	7	18

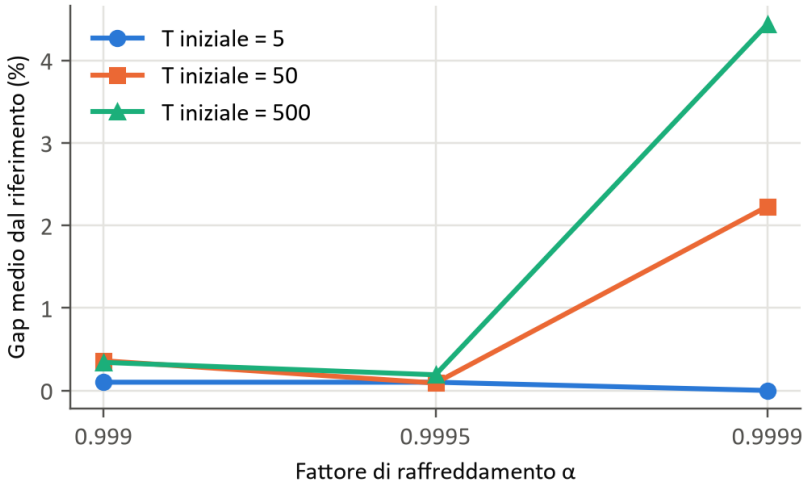


Figura 4: gap medio dal costo di riferimento al variare dei parametri.

Con una temperatura iniziale alta e un raffreddamento lento la ricerca resta quasi casuale per gran parte delle 20.000 mosse (gap fino al 4,4%). Con temperatura iniziale 5 e $\alpha = 0{,}9999$ il Simulated Annealing raggiunge il costo di riferimento in tutte le 18 prove: è la configurazione adottata. Il riferimento è un ottimo dimostrato solo in 3 giornate su 6 (nelle altre è il miglior costo trovato dal Branch and Bound); inoltre le 6 giornate fanno parte delle 26 valutate sopra, per cui il gap dello 0,08% può essere lievemente ottimistico.

Ricerca: ordine di visita e percorsi stradali

Sommario

Assegnati i clienti, per ogni veicolo restano due problemi di ricerca (`src/search/`). Il primo è l'ordine di visita: lo spazio degli stati è fatto di sequenze parziali di clienti, il costo è dato dai minuti di guida stimati più 10 volte i minuti di ritardo rispetto alle finestre effettive. Il secondo è il percorso stradale di ogni tratta, cercato con A*[23][24] sul grafo della città, dove lo stato è un nodo e il costo è il tempo di percorrenza. Il furgone parte dal deposito, non prima delle 08:00, in modo da arrivare all'apertura della prima finestra; se arriva in anticipo a un cliente successivo attende.

Strumenti utilizzati

Implementazione propria di A* e della ricerca a costo minimo (LCFS[25]) su lista di adiacenza, con frontiera a coda di priorità e potatura dei cammini multipli[26]. L'ordine di visita è cercato con un Branch and Bound in profondità: poiché i costi d'arco sono non negativi, il costo del prefisso è un limite inferiore e i prefissi che superano la migliore sequenza trovata vengono scartati. Oltre 9 clienti per veicolo si ordina per inizio della finestra. Il grafo è estratto dall'XML OSM con xml.etree.

Decisioni di Progetto

- **Grafo orientato e connesso.** La prima versione ignorava i sensi unici. Ora le strade con oneway=yes/-1 e le rotatorie generano archi in un solo verso, e il grafo è ridotto ai nodi raggiungibili dal deposito e da cui si torna al deposito: due ricerche in ampiezza, una sugli archi e una sugli archi invertiti, e l'intersezione dei nodi visitati. Restano 34.282 nodi e 58.372 archi (il 96% dei nodi percorribili): ogni coppia di nodi è collegata in entrambi i versi e nessun cliente finisce su un tratto isolato.
- **Euristica.** h(n) = distanza in linea d'aria da n alla destinazione divisa per 90 km/h, la velocità massima del grafo. Nessun arco si percorre più velocemente, quindi h non sovrastima il tempo residuo (è ammissibile); per la disuguaglianza triangolare è anche consistente, e A* con potatura dei cammini multipli resta ottimale.
- **Vincoli dalla KB.** Velocità per tipo di strada e tipi vietati dipendono dal veicolo. Prima di ogni tratta si chiede alla KB se il veicolo può trovarsi in ZTL all'orario di partenza; se non può, i nodi della ZTL diventano non attraversabili, salvo partenza e destinazione. Se una tratta non è percorribile senza ZTL (per esempio l'uscita dopo una consegna) viene percorsa comunque e il transito è registrato tra le metriche.

Valutazione

A* e LCFS sono stati confrontati su 196 coppie origine-destinazione, metà tra clienti e deposito e metà tra nodi casuali, senza vincoli e con il profilo di un furgone grande che deve evitare la ZTL (Tabella 13, Figura 5).

Tabella 13: A e LCFS sulle stesse 196 coppie origine-destinazione.*

Profilo	Algoritmo	Nodi espansi	Tempo (ms)	Costo (min)
senza vincoli	A*	8594.10 ± 7261.49	22.38 ± 19.18	7.28 ± 3.90
senza vincoli	LCFS	14543.05 ± 10208.00	19.62 ± 14.57	7.28 ± 3.90
senza vincoli	costi diversi A*/LCFS	0		
senza vincoli	nodi A* / nodi LCFS	0.59		
furgone grande (KB) + evita ZTL	A*	7801.28 ± 6455.87	20.43 ± 17.23	7.69 ± 4.04
furgone grande (KB) + evita ZTL	LCFS	13295.57 ± 9252.28	17.97 ± 13.19	7.69 ± 4.04

furgone grande (KB) + evita ZTL	costi diversi A*/LCFS	0		
furgone grande (KB) + evita ZTL	nodi A* / nodi LCFS	0.59		

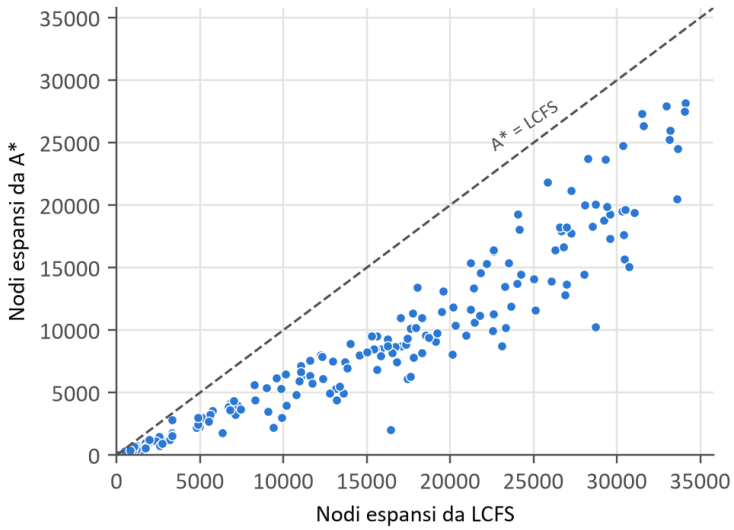


Figura 5: nodi espansi da A e da LCFS per ciascuna coppia: i punti sotto la diagonale sono coppie in cui A* espande meno nodi.*

Su tutte le coppie i due algoritmi trovano percorsi di costo identico, verifica empirica dell'ottimalità di A*, che espande in media il 41% di nodi in meno; il risparmio cresce con la lunghezza del percorso. Il tempo di A* resta comunque leggermente superiore, perché ogni espansione calcola l'euristica con funzioni trigonometriche: su un grafo di queste dimensioni il risparmio di nodi non compensa ancora il costo per nodo, e un'evoluzione possibile è precalcolare le distanze. Con i vincoli del furgone grande e l'evitamento della ZTL i percorsi si allungano in media del 6% in tempo. Il rapporto tra distanza stradale e linea d'aria (mediana 1,43) è il dato usato per fissare il fattore 1,4 delle distanze stimate nel CSP.

Valutazione del sistema integrato

Per valutare il sistema nel suo insieme, gli ordini di ciascuna delle 26 giornate sono stati ripianificati con la pipeline completa in tre modi: alla data reale, come se cadessero di domenica e come se cadessero il 7 maggio, giorno del corteo storico di San Nicola. Le metriche dei giri sono misurate dopo la simulazione con A* (Tabella 14).

Tabella 14: le stesse giornate pianificate in tre tipi di giorno (media ± dev. std. su 26 giornate).

Scenario	Clienti non servibili	Veicoli usati	km totali (A*)	Consegne in ritardo	Giri oltre turno	Transiti ZTL obbligati
reale (data storica)	0.00 ± 0.00	3.46 ± 0.97	51.07 ± 13.28	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
festivo (domenica)	0.00 ± 0.00	3.46 ± 0.97	51.07 ± 13.28	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
evento (7 maggio)	1.54 ± 1.18	3.08 ± 0.96	45.59 ± 13.48	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00

Dopo la simulazione nessuna consegna risulta in ritardo, nessun giro supera il turno e nessuna tratta richiede di attraversare la ZTL senza averne diritto: su queste istanze il rilassamento H2 non ha prodotto piani irrealizzabili. Il giorno festivo dà gli stessi piani del giorno reale, perché i clienti in ZTL sono comunque assegnati al Citan, che ha il permesso, e la fascia festiva senza ZTL (08:00-10:00) non cambia l'ottimo. Nel giorno dell'evento la KB esclude in media 1,5 clienti per giornata, quelli in ZTL con finestre fuori dalla fascia di carico e scarico, e i piani usano meno veicoli e meno chilometri.

Infine è stato pianificato il carico massimo, tutti i 33 clienti di Bari, in tre date (Tabella 15).

Tabella 15: piano dei 33 clienti di Bari in tre tipi di giorno (python src/main.py).

Scenario	Tipo giorno	Clienti serviti	Veicoli	km	Ritardi	Ultimo rientro	Giri oltre turno	Transiti ZTL obbligati
feriale	feriale	33/33	5	84.2	0	16:51	0	0
festivo	festivo	33/33	5	84.2	0	16:51	0	0
evento	evento	29/33	5	80.4	0	16:50	0	0

Anche con tutti i clienti i giri restano entro il turno, con l'ultimo rientro alle 16:51, e la KB segnala 6 clienti da preavvisare per l'alto rischio di assenza stimato dal modello.

Conclusione

Il caso di studio mostra come la pianificazione dei giri di una flotta urbana possa essere affrontata integrando rappresentazione, apprendimento, vincoli e ricerca, con la conoscenza che passa da un modulo all'altro: la KB fornisce domini e finestre al CSP e vincoli di transito ad A*, l'apprendimento aggiunge fatti alla KB, la ricerca interroga la KB tratta per tratta.

Le valutazioni indicano che lo storico del cliente è l'informazione predittiva decisiva (errore medio di 2,0 minuti contro 6,9 della baseline; F1 0,56 e AUC 0,94 sulle mancate consegne), che il Branch and Bound dimostra l'ottimo solo sulle istanze piccole (fino a circa 10 clienti) mentre il Simulated Annealing tarato resta entro lo 0,1% dalla migliore soluzione nota con tempi lineari, e che A* è ottimale con il 41% di nodi espansi in meno della ricerca a costo minimo. I limiti principali derivano dai dati: indirizzi fittizi, quindi posizioni sintetiche per macrozona, e assenza di pesi dei colli e di tempi di percorrenza osservati.

Per un uso reale in azienda gli sviluppi più utili sarebbero:

- indirizzi reali e dati di domanda (peso, volume e temperatura dei colli), per aggiungere vincoli di capacità e di catena del freddo: portata, volume e cella frigo dei veicoli sono già fatti della KB;
- la stima dei chilometri e del carburante risparmiati rispetto ai giri effettivamente percorsi, misurabile solo con posizioni e percorsi reali;
- un vincolo H2 più stretto (una stima della durata del giro) e un Branch and Bound più efficiente (bound più stretto, ordine dinamico delle variabili, esclusione degli assegnamenti equivalenti dovuti ai veicoli identici);
- tempi di percorrenza dipendenti dall'orario, appresi da dati di traffico reali e usati come pesi di A*;
- un modello probabilistico del rischio di assenza (ad esempio una rete bayesiana) che renda esplicite le dipendenze tra cliente, giorno e fascia oraria.

Riferimenti Bibliografici

[1] OpenStreetMap contributors. Estratto della rete stradale di Bari (dati del 20/07/2026), licenza ODbL. https://www.openstreetmap.org

[2] N. Fanizzi. Ingegneria della Conoscenza, dispense del corso. Dipartimento di Informatica, Università degli Studi di Bari Aldo Moro, A.A. 2025/26.

[3] D. L. Poole, A. K. Mackworth. Artificial Intelligence: Foundations of Computational Agents, 3ª ed. Cambridge University Press, 2023. https://artint.info/3e/html/ArtInt3e.html

[4] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §16.2 Classes and Properties. https://artint.info/3e/html/ArtInt3e.Ch16.S2.html

[5] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §15.4 Datalog: A Relational Rule Language. https://artint.info/3e/html/ArtInt3e.Ch15.S4.html

[6] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §5.7 Complete Knowledge Assumption. https://artint.info/3e/html/ArtInt3e.Ch5.S7.html

[7] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §5.5 Knowledge-Level Debugging. https://artint.info/3e/html/ArtInt3e.Ch5.S5.html

[8] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §5.6 Proving by Contradiction. https://artint.info/3e/html/ArtInt3e.Ch5.S6.html

[9] J. Wielemaker, T. Schrijvers, M. Triska, T. Lager. SWI-Prolog. Theory and Practice of Logic Programming 12(1-2):67-96, 2012. https://www.swi-prolog.org

[10] Y. Tekol et al. PySwip: interfaccia Python per SWI-Prolog. https://github.com/yuce/pyswip

[11] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12:2825-2830, 2011.

[12] L. Breiman. Random Forests. Machine Learning 45(1):5-32, 2001.

[13] J. H. Friedman. Greedy Function Approximation: A Gradient Boosting Machine. The Annals of Statistics 29(5):1189-1232, 2001.

[14] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §7.5 Composite Models. https://artint.info/3e/html/ArtInt3e.Ch7.S5.html

[15] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §7.4 Overfitting. https://artint.info/3e/html/ArtInt3e.Ch7.S4.html

[16] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §7.2 Supervised Learning Foundations. https://artint.info/3e/html/ArtInt3e.Ch7.S2.html

[17] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §4.8 Optimization. https://artint.info/3e/html/ArtInt3e.Ch4.S8.html

[18] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §4.3 Consistency Algorithms. https://artint.info/3e/html/ArtInt3e.Ch4.S3.html

[19] A. K. Mackworth. Consistency in Networks of Relations. Artificial Intelligence 8(1):99-118, 1977.

[20] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §4.4 Domain Splitting. https://artint.info/3e/html/ArtInt3e.Ch4.S4.html

[21] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §4.6 Local Search. https://artint.info/3e/html/ArtInt3e.Ch4.S6.html

[22] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi. Optimization by Simulated Annealing. Science 220(4598):671-680, 1983.

[23] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §3.6 Informed (Heuristic) Search. https://artint.info/3e/html/ArtInt3e.Ch3.S6.html

[24] P. E. Hart, N. J. Nilsson, B. Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics 4(2):100-107, 1968.

[25] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §3.5 Uninformed Search Strategies. https://artint.info/3e/html/ArtInt3e.Ch3.S5.html

[26] D. L. Poole, A. K. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3ª ed., §3.7 Pruning the Search Space. https://artint.info/3e/html/ArtInt3e.Ch3.S7.html